Apache Kafka Lab

Experiment 1: Install Kafka on a Single Node
Experiment 2: Single-Node, Single-Broker Kafka Setup with Python Producer/Consumer

Software Needed

1. Docker Desktop

Mac: https://www.docker.com/products/docker-desktop/

Windows: https://www.docker.com/products/docker-desktop/

2. Python 3

Check version:

Mac

```
python3 --version
```

Windows

```
python --version
```

3. VS Code (optional but recommended)

Download: https://code.visualstudio.com/

Experiment 1: Install Apache Kafka on a Single Node

Objective: Install and run Apache Kafka on a single-node system using Docker. The following commands are identical on Mac Terminal and Windows PowerShell.

Step 1: Check Docker Installation

```
docker --version
docker compose version
```

Step 2: Start Kafka

```
docker run -d --name kafka -p 9092:9092 apache/kafka:4.3.1
```

Step 3: Check Whether Kafka Is Running

```
docker ps
```

Step 4: Check Kafka Logs

```
docker logs kafka
```

```
docker logs -f kafka   (Ctrl + C to exit)
```

Step 5: Stop / Start / Remove Kafka

```
docker stop kafka
docker start kafka
docker rm -f kafka
```

Experiment 1 Result

Should demonstrate the following command and show that the Kafka container is running:

```
docker ps
```

Viva Questions for Experiment 1

1. What is Apache Kafka?
2. What is a Kafka broker?
3. What is a Kafka topic?
4. Why are we using Docker?
5. What is a single-node Kafka setup?
6. Which port does Kafka commonly use?

Answer: 9092

Experiment 2: Single-Node, Single-Broker Kafka Setup + Python Producer/Consumer

Step 1: Create a Project Folder

Mac (Terminal)

```
mkdir kafka-lab
cd kafka-lab
```

Windows (PowerShell)

```
mkdir kafka-lab
cd kafka-lab
```

Step 2: Create Python Virtual Environment

Mac

```
python3 -m venv venv
source venv/bin/activate
```

Windows

```
python -m venv venv
venv\Scripts\activate
```

Step 3: Install Kafka Python Library

```
pip install kafka-python
pip show kafka-python
```

Step 4: Start Kafka (if not already running)

```
docker start kafka
docker ps
```

Step 5: Create a Topic Using Python

Create a file named create_topic.py:

```
from kafka.admin import KafkaAdminClient, NewTopic

admin_client = KafkaAdminClient(
    bootstrap_servers="localhost:9092",
    client_id="python-admin"
)

topic = NewTopic(
    name="college-topic",
    num_partitions=1,
    replication_factor=1
)

try:
    admin_client.create_topics(new_topics=[topic])
    print("Topic created successfully!")
except Exception as e:
    print("Topic may already exist:", e)

admin_client.close()
```

Run:

Mac

```
python3 create_topic.py
```

Windows

```
python create_topic.py
```

Expected output:

```
Topic created successfully!
```

Step 6: List Topics Using Python

Create a file named list_topics.py:

```
from kafka import KafkaConsumer

consumer = KafkaConsumer(
```

```
    bootstrap_servers="localhost:9092"
)

topics = consumer.topics()

print("Available Topics:")
for topic in topics:
    print(topic)

consumer.close()
```

Run:

Mac

```
python3 list_topics.py
```

Windows

```
python list_topics.py
```

Expected output:

```
college-topic
```

Step 7: Python Producer

Create a file named producer.py:

```
from kafka import KafkaProducer

producer = KafkaProducer(
    bootstrap_servers="localhost:9092"
)

messages = [
    "Hello Kafka",
    "This is a Python producer",
    "Welcome to ETL Lab"
]

for message in messages:
    producer.send(
        "college-topic",
        message.encode("utf-8")
    )
    print("Sent:", message)

producer.flush()

print("All messages sent successfully!")

producer.close()
```

Run:

Mac

```
python3 producer.py
```

Windows

```
python producer.py
```

Expected output:

```
Sent: Hello Kafka
Sent: This is a Python producer
Sent: Welcome to ETL Lab
All messages sent successfully!
```

Step 8: Python Consumer

Create a file named consumer.py:

```
from kafka import KafkaConsumer

consumer = KafkaConsumer(
  "college-topic",
  bootstrap_servers="localhost:9092",
  auto_offset_reset="earliest",
  enable_auto_commit=True,
  group_id="college-group"
)

print("Waiting for messages...")

for message in consumer:
  print(
    "Received:",
    message.value.decode("utf-8")
  )
```

Run:

Mac

```
python3 consumer.py
```

Windows

```
python consumer.py
```

Expected output:

```
Waiting for messages...

Received: Hello Kafka
Received: This is a Python producer
Received: Welcome to ETL Lab
```

Stop the consumer using:

```
Ctrl + C
```

Summary to Demonstrate Experiment 2 in short

Open three terminal windows and use the OS-appropriate command for each:

Terminal 1 — Kafka

```
docker ps
```

Terminal 2 — Consumer

Mac

```
python3 consumer.py
```

Windows

```
python consumer.py
```

Terminal 3 — Producer

Mac

```
python3 producer.py
```

Windows

```
python producer.py
```

Recommended Folder Structure

```
kafka-lab/
│
├── venv/
├── create_topic.py
├── list_topics.py
├── producer.py
└── consumer.py
```

So the Experiment 2 demonstration should follow this flow:

```
Start Kafka
  ↓
Create Topic using Python
  ↓
Run Python Consumer
  ↓
Run Python Producer
  ↓
Consumer receives messages
```

Experiment 1

- Install Docker
- Run Kafka container

- Verify using docker ps

Experiment 2

- Install kafka-python
- Create topic using Python
- Create Python producer
- Create Python consumer
- Demonstrate message flow

Recommended Environment Summary

Component	Technology
Operating System	Mac / Windows
Kafka Installation	Docker
Kafka Mode	Single Node
Broker	Single Broker
Programming Language	Python
Python Library	kafka-python
IDE	VS Code
Kafka Port	9092

References

Apache Kafka Docker documentation: https://kafka.apache.org/43/getting-started/docker/
kafka-python on PyPI: https://pypi.org/project/kafka-python/